

5 Remaining Specs: Forgetting, Causal, Temporal, Imagination, Recall

Attention quality fix verified. V5 Phase 1 complete. These are the exact instructions for Phase 2-3.

ATTENTION QUALITY FIX — VERIFIED

Commit a044a4a. All 4 round-18 issues closed:

1. narrative field: ADDED. "Your organization's attention is well-distributed."

2. "unknown" domain: FILTERED. Only 5 real domains (payments, auth, platform, deployment, incident).

3. Percentages: NORMALIZED. current=100%, recommended=100% (was 136%).

4. Redistribution: WORKS. Caps at 35%, redistributes proportionally. (No redistribution needed with demo data — honest.)

V5 Phase 1: COMPLETE. Spec #1 (hide organs) DELIVERED. Spec #2 (executive function) DELIVERED. Spec #3 (attention) DELIVERED. 389 tests pass, 0 fail.

Build Order (Critical — Dependencies Matter)

The 5 specs have dependencies. Build them in this exact order. Do NOT skip ahead.

Step	Spec	Why This Order	Effort
1	#7 Temporal Trajectories	Foundation for #5 (Imagination) and V6 (Trajectory Intervention)	1. Builds on consciousness.py + time_axis.py
2	#6 Causal Cognition	Foundation for #5 (Imagination needs causal models for counterfactuals)	2. Builds on prediction_lifecycle.py +
3	#4 Forgetting Engine	Independent. Builds on learning.py + evidence_graph.py. Can be built in parallel with #6.	1.5 days
4	#5 Imagination (Counterfactual)	Depends on #6 (causal) + #7 (temporal). Builds on digital_twin.py	2 days. Counterfactuals need causal chains.
5	#8 Institutional Recall	Depends on #7 (temporal history). Builds on learning.py + signal history.py. Enhanced by #6 (causal context).	1.5 days
TOTAL	5 specs	Build in order: #7 → #6 → #4 (parallel with #6) → #5 → #8	9 days

The V5 litmus test applies to ALL specs: "Does this make Maestro feel simpler?" Every spec must be INVISIBLE from the start — no new panels, no new organ names, no new sidebar items. Each spec enhances an EXISTING surface (TODAY, LEARN, Cognition, or ASK). If a spec adds a new visible panel, it fails the litmus test — even if the backend is brilliant.

Step 1: Spec #7 — Temporal Trajectories (Build First)

SPEC #7 Temporal Trajectories — "Trust has fallen slowly for 8 weeks"

V5 Principle

V5: "Don't remember snapshots. Remember trajectories." Today, `consciousness.py` returns a point-in-time state vector. V5 extends this to org-wide trajectory memory: every consciousness dimension (attention, knowledge, trust, conflict, energy, uncertainty, learning) becomes a trajectory — current value + trend + slope + duration + narrative.

Current codebase gap

`consciousness.py` (204 lines) computes 7 dimensions as point-in-time values. `time_axis.py` (167 lines) returns past/present/future for a single domain. The gap: no org-wide trajectory tracking. Maestro can say "trust is currently 0.42" but not "trust has fallen for 8 weeks." The trajectory — the trend over time — is not tracked for organizational dimensions.

Files to create/modify

CREATE `backend/maestro_oem/temporal_trajectories.py` — the `TemporalTrajectoryEngine`. Maintains a rolling 90-day trajectory for each consciousness dimension. On construction, scans signal history to build trajectory data. For each dimension: {current, trend (improving/declining/stable), slope (float), duration_weeks (int), narrative (sentence)}. CREATE GET `/api/oem/trajectories` endpoint. MODIFY `backend/maestro_oem/consciousness.py` — enrich each state dimension with its trajectory (add a `trajectory` sub-object to each dimension in `state_vector()`). MODIFY `static/js/cognition.js` — the "Right now" section (was `Consciousness`) now shows trajectory sentences: "Trust between Engineering and Legal has fallen for 8 weeks" instead of "Trust: 0.42".

API contract

GET `/api/oem/trajectories` returns: { "trust": { "current": 0.42, "trend": "declining", "slope": -0.03, "duration_weeks": 8, "narrative": "Trust between Engineering and Legal has fallen slowly for 8 weeks." }, "attention": { "current": 0.65, "trend": "stable", "slope": 0.01, "duration_weeks": 12, "narrative": "Attention distribution has been stable." }, ... 7 dimensions ... }. At least 3 dimensions must have trend != "stable" (or honest "insufficient history — need 4+ weeks of data").

Acceptance test (auditor will run)

1) GET `/api/oem/trajectories` returns 7 dimensions, each with current, trend, slope, duration_weeks, narrative. 2) Each narrative is a trajectory sentence (not a point metric). 3) Dimensions with < 4 weeks of data return "insufficient history" honestly. 4) Cognition surface "Right now" section shows trajectory sentences. 5) No hardcoded trajectories — all computed from signal history. 6) V5 litmus: no new panel — enhances existing Cognition surface.

Effort

1.5 days

Dependencies

None (foundation spec — build first). Builds on `consciousness.py` + signal history.

Step 2: Spec #6 — Causal Cognition (Build Second)

SPEC #6 Causal Cognition — "A caused B because 5 interventions produced the same sequence"

V5 Principle

V5: "Today: A correlates with B. Future: We believe A caused B because five similar interventions produced the same sequence." Causal cognition moves Maestro from correlation to causation.

Current codebase gap

`evidence_graph.py` tracks co-occurrence (correlation). `prediction_lifecycle.py` (859 lines) tracks resolved predictions where an intervention was recommended and an outcome was observed. But neither builds CAUSAL CHAINS — "intervention X caused outcome Y, observed N times, failed M times." The gap: Maestro can say "bottlenecks correlate with velocity drops" but not "removing the bottleneck CAUSED velocity to recover, because 3 interventions produced the same recovery."

Files to create/modify

CREATE `backend/maestro_oem/causal.py` — the CausalEngine. Scans `prediction_lifecycle.py` database for resolved predictions where: (1) a recommendation was made (intervention), (2) the prediction was resolved as hit or miss (outcome), (3) the same intervention-outcome pair occurred ≥ 3 times. Builds causal chains: {intervention, effect, sequence_count, failed_count, confidence, first_observed, evidence}. A causal claim requires `sequence_count` ≥ 3 AND `failed_count` == 0 (or a clear failure ratio). CREATE GET `/api/oem/causal?intervention=...` endpoint. MODIFY `backend/maestro_oem/wisdom.py` — when synthesizing judgment, cite causal chains where available ("This intervention caused this outcome 3 times with 0 failures"). MODIFY `static/js/cognition.js` — the "When values compete" section (was Wisdom) now shows "Why we believe this" with causal evidence.

API contract

GET `/api/oem/causal?intervention=redistribute+approval+authority` returns: { "intervention": "Redistribute approval authority", "causes": [{"effect": "Approval time drops below 2 days", "sequence_count": 3, "failed_count": 0, "confidence": 0.91, "evidence": ["rec-xxx: bottleneck addressed → velocity recovered in 5 days"]}], "narrative": "Redistributing approval authority causes approval time to drop. Observed 3 times, 0 failures." }. If insufficient data: { "causes": [], "narrative": "Insufficient intervention data for causal inference. The pilot needs more resolved predictions." }.

Acceptance test (auditor will run)

1) GET `/api/oem/causal` returns causes array (or honest "insufficient data"). 2) Each cause has `sequence_count` ≥ 3 , `failed_count`, `confidence`. 3) Narrative is a causal claim (not correlation). 4) Cognition "When values compete" section cites causal evidence. 5) No hardcoded causal claims. 6) V5 litmus: no new panel — enhances existing Wisdom section.

Effort

2 days

Dependencies

Spec #7 (Temporal) for temporal context. Builds on `prediction_lifecycle.py` + `evidence_graph.py`.

Step 3: Spec #4 — Forgetting Engine (Build Third, Can Parallel #6)

SPEC #4 Forgetting Engine — archive zero-predictive-value events

V5 Principle

V5: "Brains forget for a reason. Compression is not forgetting. Without forgetting, memory eventually becomes noise." The Forgetting Engine identifies events with zero future predictive value and archives them.

Current codebase gap

memory_compression.py (152 lines) compresses experience into truths/habits/mistakes. But it does not forget — all signals remain in the active evidence graph. A 2-year-old incident that never correlated with any future outcome still occupies cognitive space. The gap: no mechanism to identify and archive zero-predictive-value events.

Files to create/modify

CREATE backend/maestro_oem/forgetting.py — the ForgettingEngine. For each learning object and signal in the active model, compute predictive_value (0.0-1.0) based on: (1) has it been referenced in any recent recommendation? (2) has it correlated with any future outcome? (3) has it been validated/invalidated? Events with predictive_value < 0.05 AND age > 180 days are archive candidates. The engine does NOT delete — it flags candidates for archiving (cold storage). CREATE GET /api/oem/forgetting endpoint. MODIFY backend/maestro_oem/engine.py — on ingest, check if new signals make old events irrelevant (predictive_value drops). MODIFY static/js/cognition.js — the "What it all comes down to" section (was Memory Compression) now shows a "Maestro is forgetting" line: "3 old events have produced no predictive value. Maestro is quietly archiving them."

API contract

GET /api/oem/forgetting returns: { "archive_candidates": [{"entity_id": "...", "type": "signal", "age_days": 245, "predictive_value": 0.02, "reason": "No correlation with any future outcome in 245 days"}], "archived_count": 0, "active_working_set": 1247, "threshold": 0.05, "narrative": "3 events have zero predictive value. Maestro is quietly archiving them to reduce cognitive noise." }. If no candidates: { "archive_candidates": [], "narrative": "All events have predictive value. Nothing to forget yet." }.

Acceptance test (auditor will run)

1) GET /api/oem/forgetting returns archive_candidates array + narrative. 2) Each candidate has predictive_value < 0.05 and age_days > 180 (or honest "nothing to forget"). 3) Engine does NOT delete (flags only). 4) Cognition surface shows "Maestro is forgetting" line. 5) No hardcoded candidates. 6) V5 litmus: no new panel — enhances existing Compression section.

Effort

1.5 days

Dependencies

None (independent — can parallel #6). Builds on learning.py + evidence_graph.py.

Step 4: Spec #5 — Imagination / Counterfactual (Build Fourth)

SPEC #5 Imagination (Counterfactual) — "What would happen if Legal disappeared?"

V5 Principle

V5: "Organizations don't only solve today's problems. They imagine futures." The Imagination organ performs counterfactual reasoning: given a hypothetical change, predict the organizational impact using causal models and historical data.

Current codebase gap

digital_twin.py (743 lines) runs scenarios (person leaves, team doubles) via parametric simulation. But these are PARAMETRIC — they adjust inputs and compute outputs. They are not COUNTERFACTUAL — they do not reason about causality ("Legal disappeared BECAUSE of the reorg, and the reorg also affected Engineering, so..."). The gap: no causal counterfactual reasoning.

Files to create/modify

CREATE backend/maestro_oem/imagination.py — the ImaginationEngine. Takes a counterfactual scenario ("What would happen if Legal disappeared?") and: (1) identifies causal dependencies of the entity (from causal.py + evidence_graph.py), (2) retrieves historical analogues (has any team ever been restructured?), (3) simulates the impact using digital_twin.py, (4) produces a causal narrative. CREATE GET /api/oem/imagine?scenario=... endpoint. MODIFY static/js/ask_v2.js — when the user asks a "what if" question, route to the Imagination engine and render the counterfactual narrative.

API contract

GET /api/oem/imagine?scenario=What+would+happen+if+Legal+disappeared returns: { "scenario": "Legal disappeared", "consequences": [{"effect": "OAuth reviews would stall", "cause": "Legal is the sole reviewer", "evidence_count": 5, "confidence": 0.82}], "historical_analogue": "The Q2 legal-team reorg produced similar dynamics.", "narrative": "If Legal disappeared, immediate effect: faster engineering. Secondary effect: more incidents within 3 weeks." }. Must have 2+ consequences with cause + evidence_count + confidence, and 1 historical analogue.

Acceptance test (auditor will run)

1) GET /api/oem/imagine returns consequences (2+) + historical_analogue + narrative. 2) Each consequence has cause + evidence_count + confidence. 3) Historical analogue references real data. 4) ASK v2 routes "what if" questions to Imagination. 5) No hardcoded scenarios. 6) V5 litmus: no new panel — enhances existing ASK v2.

Effort

2 days

Dependencies

Spec #6 (Causal) + Spec #7 (Temporal). Builds on digital_twin.py. Counterfactuals need causal chains.

Step 5: Spec #8 — Institutional Recall (Build Last)

SPEC #8 Institutional Recall — "When have we been here before?"

V5 Principle

V5: "Eventually Maestro owns the organization's autobiography." Institutional Recall answers "When have we been here before?" with 3 moments + lessons — not documents, not search results, but organizational memories.

Current codebase gap

The autocomplete engine returns suggestions, not memories. learning.py tracks calibration but not narrative recall. memory_compression.py compresses but does not retrieve by situational similarity. The gap: no mechanism to take a current situation and retrieve the most similar past moments with their outcomes and lessons.

Files to create/modify

CREATE backend/maestro_oem/institutional_recall.py — the InstitutionalRecallEngine. Takes a situation description (from current recommendation or ASK query) and retrieves top 3 similar past moments from: learning objects, resolved predictions, contradiction history, signal history. Uses keyword overlap (or embedding if available) against the historical database. For each moment: {when, situation, what_we_did, what_we_learned, outcome}. CREATE GET /api/oem/recall?situation=... endpoint. MODIFY static/js/ask_v2.js — append a "When you've been here before" section with up to 3 recalled moments.

API contract

GET /api/oem/recall?situation=delaying+launch+for+Legal+review returns: { "moments": [{"when": "2024-08-15", "situation": "Q3 auth launch delayed 3 days for Legal review", "what_we_did": "Proceeded with Legal review", "what_we_learned": "Legal review before launch reduces post-launch incidents", "outcome": "succeeded", "evidence_count": 5}], "summary": "You've been in a similar situation 3 times. In 2 of 3, proceeding with the Legal review led to better outcomes." }. If novel: { "moments": [], "summary": "This situation is novel — no similar past moments found." }.

Acceptance test (auditor will run)

1) GET /api/oem/recall returns moments array (or honest "novel situation"). 2) Each moment has when, situation, what_we_did, what_we_learned, outcome. 3) Moments reference real historical data. 4) ASK v2 shows "When you've been here before" section. 5) Summary is a narrative. 6) V5 litmus: no new panel — enhances existing ASK v2.

Effort

2 days

Dependencies

Spec #7 (Temporal) for historical context. Builds on learning.py + signal history + prediction_lifecycle.py.

Strict Rules for All 5 Specs

Rule	Enforcement
1. Build in order: #7 -> #6 -> #4 (parallel with #6) -> #5	Dependencies are real. #5 needs #6. #8 benefits from #7. Do not skip ahead.
2. Every spec enhances an EXISTING surface	#7 -> Cognition "Right now". #6 -> Cognition "When values compete". #4 -> Cognition "What it all comes down to"
3. V5 litmus test: UI must be SIMPLER after each spec	No new sidebar items (stays at 4). No new organ names. No new jargon. Each spec makes an existing section richer
4. Every API must return REAL data (not hardcoded)	Grep the backend module for hardcoded values. Every response field must be computed from model data. The model must be real
5. Honest degradation when data is insufficient	If causal.py finds < 3 interventions: return "Insufficient intervention data for causal inference." If recall finds no data: return "No data for recall"
6. Every spec must pass the 5-point checklist	1) Ran FULL acceptance test (API + frontend)? 2) Checked APPLICATION? 3) Ran FULL test suite? 4) Verified litmus test? 5) Reported exact pytest output?
7. Frontend text passes through humanize()	All user-facing strings from API responses must be passed through humanize() before rendering. No law codes, no raw JSON
8. No silent skips	If a spec is deferred, say so explicitly in the commit message. Do NOT claim "all 5 specs delivered" if only 4 are delivered
9. Run the FULL test suite (not a subset)	python -m pytest backend/maestro_api/tests/ backend/maestro_auth/tests/ backend/maestro_oem/tests/ --tb=short
10. CI pipeline must pass	The .github/workflows/test.yml CI pipeline runs on every push. It must pass. If it fails, fix before claiming delivery

Anti-Pattern Warning (Read Twice)

THE PATTERN WAS BROKEN IN ROUND 16. MAINTAIN IT FOR V5 PHASE 2-3.

The "built but not applied" pattern recurred 6 times (rounds 7-13) and was broken in round 16 (all 8 V4 organs wired). V5 Phase 1 maintained the break (round 18: all 3 specs wired, no new panels). V5 Phase 2-3 must maintain it again.

The specific risk for Phase 2-3: Specs #4 (Forgetting), #6 (Causal), and #7 (Temporal) are backend-heavy. The temptation is to build the engine, verify via API, and claim delivery without wiring the frontend. **DO NOT DO THIS.** Every spec must enhance an existing frontend surface (Cognition or ASK v2) before it is claimed. The acceptance test requires: "Auditor opens [surface] and verifies [user-visible output]." If the frontend is not wired, the spec is NOT delivered — regardless of whether the API returns 200.

The "added visibility" risk: V5 Phase 1 successfully avoided adding new panels. Phase 2-3 must do the same. Specs #4, #6, #7 enhance the Cognition surface (existing). Specs #5, #8 enhance ASK v2 (existing). If any spec adds a new sidebar item, a new surface, or a new panel, it fails the V5 litmus test. The sidebar stays at 4 items. No exceptions.

The "subset test reporting" risk: The coder has reported subset test counts as full-suite counts twice (rounds 3 and 8). The CI pipeline now prevents this structurally. But the coder must also run the full suite locally before claiming delivery. Report the exact `pytest` output line. Do not round. Do not subset.

Summary — What "Done" Looks Like

Spec	Backend	API	Frontend	Litmus Test	Done When
#7 Temporal	temporal_trajectories.py	GET /trajectories	Cognition "Right now" shows trajectory sentences	No new panel	7 dimensions with trend + narrative in Cognition
#6 Causal	causal.py	GET /causal?intervention=...	Cognition "When values compete" cites causal chains	No new panel	Causal chains with sequence_count >= 3 (or honest)
#4 Forgetting	forgetting.py	GET /forgetting	Cognition "What it all comes down to" shows MAESTRO	No new panel	Archive candidates with predictive_value < 0.05
#5 Imagination	imagination.py	GET /imagine?scenario=...	ASK v2 routes "what if" questions to Imagination	No new panel	2+ consequences with cause + confidence + history
#8 Recall	institutional_recall.py	GET /recall?situation=...	ASK v2 shows "When you've been here before"	No new panel	Moments with when/situation/what_we_did/what_we_

When all 5 specs are done: V5 is complete (Phase 1 + Phase 2 + Phase 3). The Chrome extension (#9) already exists (183 lines content.js). The product is The Invisible Layer — cognitive organs that are invisible, executive function that acts, attention that allocates, forgetting that reduces noise, causal reasoning that explains why, temporal trajectories that show where things are heading, imagination that counterfactuals, and institutional recall that remembers when you've been here before. All invisible. All enhancing existing surfaces. The UI is simpler than when we started. The intelligence is deeper. That is V5. Build it.